Cloud Services and Architecture on AWS

Name: Anusiya R

Program: VertexMind Internship, Mission 2: Cloud Services & Architecture

Date: 26 October 2026

1. Introduction

In Mission 1, I learned cloud basics and hosted a static website on Amazon S3. In Mission 2, I worked with three core cloud services: storage, a database and serverless computing. I created each service on AWS in the Asia Pacific (Sydney) region, tested it, and designed an architecture diagram showing how these services fit together.

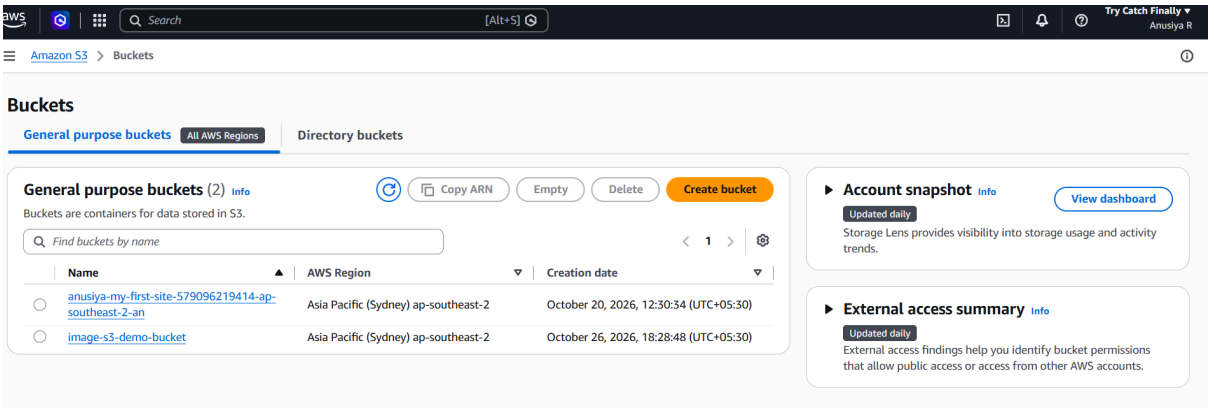
2. Storage: Amazon S3

Amazon S3 (Simple Storage Service) stores files as objects inside buckets. It is durable, scalable and pay-as-you-go.

What I did:

1. Opened the S3 service and clicked Create bucket.
2. Created a general purpose bucket named `image-s3-demo-bucket`.
3. Kept Block all public access turned on, because this bucket is for private storage and not a website.

4. Uploaded a sample file to the bucket.





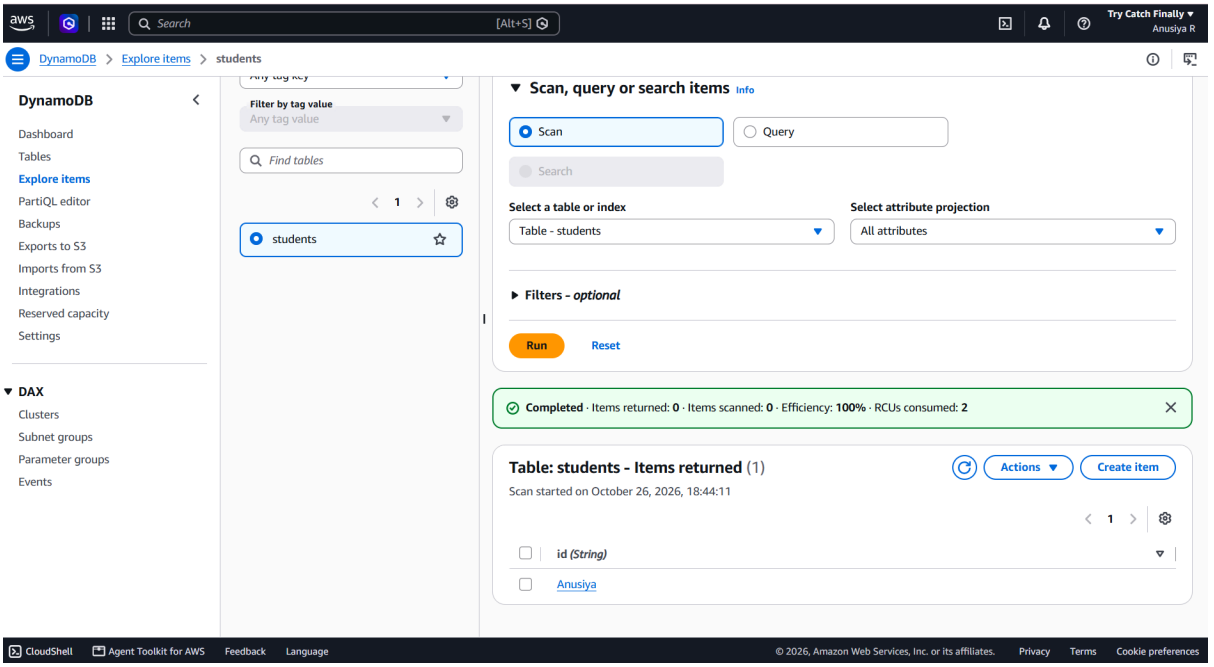
3. Database: Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database. AWS handles servers, scaling and backups, so I only create tables and store data.

What I did:

1. Opened DynamoDB and clicked Create table.
2. Named the table `students` with partition key `id` (String).
3. Kept the default settings and created the table.
4. Used Explore items → Create item to add a sample record.

5. Ran a scan to confirm the record was stored.



4. Serverless: AWS Lambda

AWS Lambda runs code without the need to manage servers. The code runs only when it is triggered, and I pay only for the execution time. This is called serverless computing.

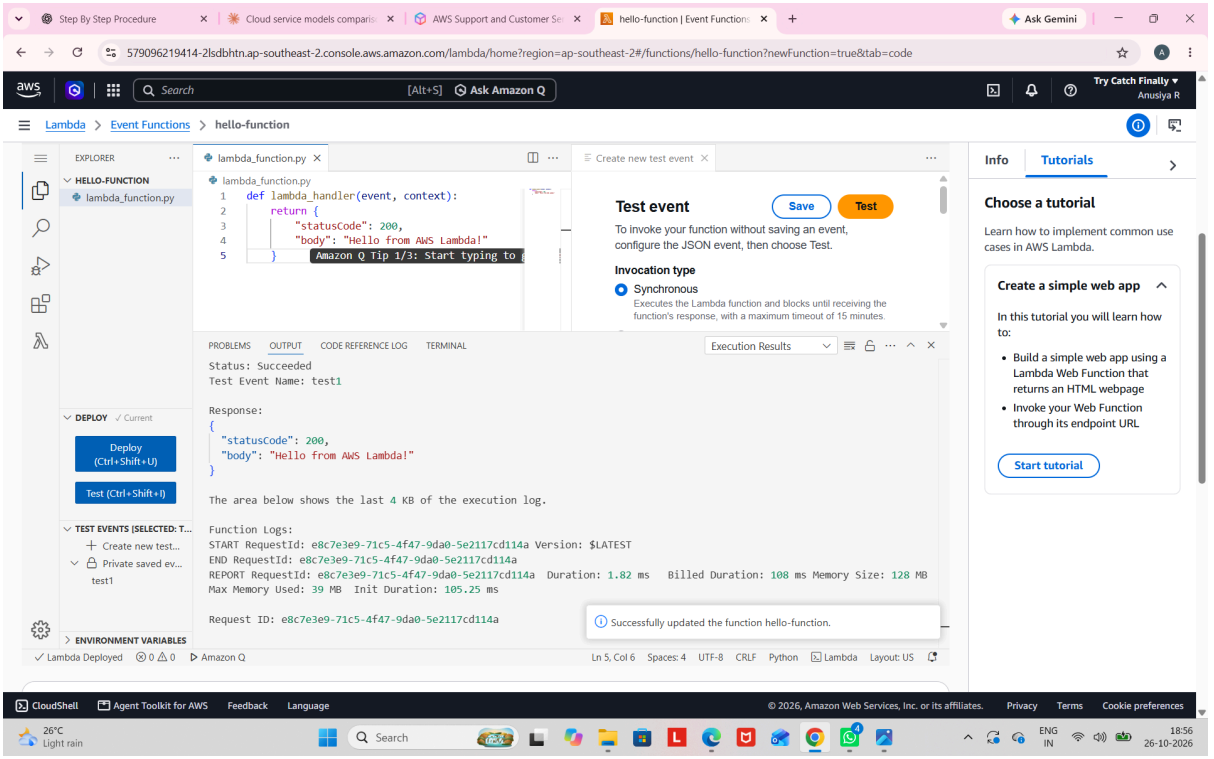
What I did:

1. Open Lambda and clicked create function → Author from scratch.
2. Named the function `hello-function` and chose a Python runtime.
3. Wrote this handler code and clicked Deploy:

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "Hello from AWS Lambda!"
    }
```

4. Created a test event named `test1` and ran the test.

5. The result was Status: Succeeded with `statusCode` 200 and the message "Hello from AWS Lambda!".



5. Architecture Diagram

The diagram below shows how a user, Lambda, S3 and DynamoDB work together in a typical cloud application.

[Insert the exported architecture-diagram.png here]

How it works:

- The **user** sends a request to the application.
- **AWS Lambda** runs the application logic without any server to manage.
- **Amazon S3** stores files such as images and documents.
- **Amazon DynamoDB** stores structured records such as student details.

In this mission I created and tested each service on its own. The diagram shows the intended architecture, where Lambda would be given permissions to read and write to S3 and DynamoDB.

6. Service Comparison

Service	Type	Purpose	Managed by AWS
Amazon S3	Object storage	Store files and media	Hardware, durability, scaling
Amazon DynamoDB	NoSQL database	Store and query records	Servers, scaling, backups
AWS Lambda	Serverless compute	Run code on demand	Servers, OS, scaling

7. Challenges and Learnings

- Lambda code must be deployed before testing, or the old code runs.
- Resources should stay in the same AWS region so they are easy to find and connect.
- Private data needs Block Public Access turned on, while public websites need it turned off.
- Serverless services such as Lambda and DynamoDB need almost no server management.
- I should delete unused resources after the mission to avoid charges.

8. Conclusion

This mission gave me hands-on experience with three important AWS services: S3 for storage, DynamoDB for databases and Lambda for serverless functions. I also learned how to describe them in an architecture diagram. These services are the building blocks of many modern cloud applications, and this experience prepares me for the next missions on cloud security and networking.